딥러닝 프로그래밍

한국폴리텍대학 하이테크과정
2026년 2학기
박병주 강사

목차

A table of Contents

01 지난시간 복습

02 Convolution

03 feature map 크기 조절

04 CNN 계통과 조립

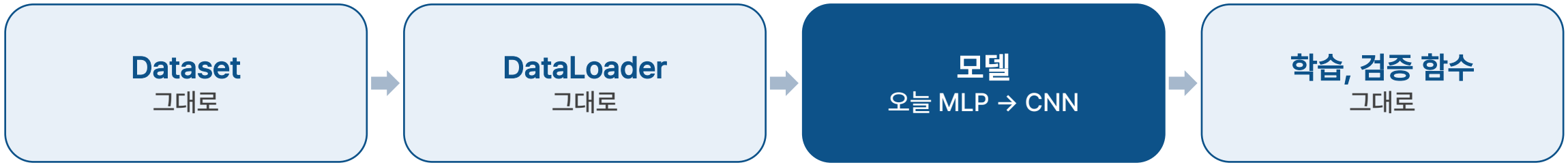
05 CNN 학습과 평가

06 CNN이 강한 이유

PART 01

지난시간 복습

1. 복습: Baseline과 실험 기록표



실험	바꾼 것	best val (epoch)	test	Baseline 대비 (val)
Baseline	없음, MLP 784-100-10	97.25% (10)	97.34%	기준
실험 1	optimizer 설정, SGD lr 0.1 → Adam lr 0.001	97.43% (10)	97.59%	+0.18%p

- 학습 함수와 검증 함수는 그대로, **모델만 CNN으로 바꿔** 실험 2로 추가
- 판단은 **검증 정확도**, test는 기록용

1. MLP는 사진을 한 줄로 펴서 본다



- MLP는 첫 계층에서 사진을 784개 숫자 한 줄로 펴고, **픽셀마다 따로** weight를 곱한다
- 어떤 픽셀이 이웃인지는 모델 구조에 남지 않는다

1. MLP로 사진을 다룰 때의 세 가지 한계

1. 이웃 관계를 모른다
사진을 한 줄로 펴서
784개를 따로 본다

2. 사진이 크면 파라미터 폭증
입력 숫자 수 × 뉴런 수만큼
weight가 필요하다

3. 위치 변화에 약하다
숫자를 몇 칸만 옮겨도
같은 자리에 다른 값이 들어온다

사진	입력 숫자 수	fc1 파라미터 (뉴런 100개)
MNIST 흑백 28×28	784	78,500
컬러 32×32	3,072	307,300
컬러 사진 224×224	150,528	**15,052,900**

fc1 파라미터 수

$$N_{\text{fc1}} = n_{\text{in}} \times 100 + 100$$

n_in은 입력 숫자 수, 100은 뉴런 수와 bias 수

- CNN은 kernel, weight sharing, pooling으로 세 가지를 줄인다, 1과 3은 실습 5와 실습 6에서 확인

1. 오늘의 목표



- **오늘 할 일 세 가지**
  - kernel로 이웃 픽셀을 함께 보는 **Convolution**과 크기를 줄이는 **Pooling**을 손으로 확인한다
  - `Conv2d`와 `MaxPool2d`로 CNN 클래스를 만들고 shape와 파라미터 수를 따라간다
  - 같은 학습 함수로 CNN을 학습해 **Baseline 97.25%**와 비교한다

1. 준비 셀 1: 데이터

준비 셀 1. 데이터, 한 줄 추가

```
import torch
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split
device = "cuda" if torch.cuda.is_available() else "cpu"
torch.backends.cudnn.deterministic = True   # 새 줄, GPU에서도 같은 seed면 같은 결과
train_all = datasets.MNIST(root="data", train=True, download=True,
                           transform=transforms.ToTensor())
g = torch.Generator().manual_seed(0)            # 학습과 검증 분리, seed 0
train_set, val_set = random_split(train_all, [50000, 10000], generator=g)
train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
val_loader = DataLoader(val_set, batch_size=256, shuffle=False)
print(len(train_loader), len(val_loader))      # → 782 40
```

- **새 줄 하나**
  - `cudnn.deterministic`은 cuDNN이 결과가 매번 같게 나오는 계산 방법만 쓰게 해, 오늘 모델에서는 **같은 seed면 같은 결과**가 나온다
  - 이 줄이 없으면 같은 seed여도 소수점이 조금씩 달라질 수 있다
- **확인**
  - `782 40`이 나오면 성공이다

1. 준비 셀 2: MLP와 loss

준비 셀 2. Baseline MLP와 loss_fn

```
class MLP(nn.Module):                       # Baseline MLP 그대로
    def __init__(self):
        super().__init__()
        self.flat = nn.Flatten()            # (N, 1, 28, 28) → (N, 784)
        self.fc1 = nn.Linear(784, 100)
        self.act = nn.ReLU()
        self.fc2 = nn.Linear(100, 10)
    def forward(self, x):
        x = self.flat(x)
        return self.fc2(self.act(self.fc1(x)))

loss_fn = nn.CrossEntropyLoss()
print(MLP()(torch.zeros(2, 1, 28, 28)).shape)   # → torch.Size([2, 10])
```

- **오늘 MLP를 다시 쓰는 곳**
  - CNN과 **파라미터 수** 비교
  - 실습 5에서 섞인 사진으로 **다시 학습**
- **확인**
  - `torch.Size([2, 10])`이 나오면 성공이다

1. 준비 셀 3: 검증 함수와 학습 함수

준비 셀 3-1. 검증 함수

```python
def evaluate(model, loader):        # 검증 함수, 그대로
    model.eval()
    total_loss, correct = 0.0, 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            out = model(x)
            total_loss += loss_fn(out, y).item() * len(y)
            correct += (out.argmax(dim=1) == y).sum().item()
    n = len(loader.dataset)
    return total_loss / n, correct / n * 100
```

준비 셀 3-2. 학습 함수

```python
def train_one_epoch(model, loader, opt):    # 학습 함수, 그대로
    model.train()
    total_loss = 0.0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        loss = loss_fn(model(x), y)
        opt.zero_grad()
        loss.backward()
        opt.step()
        total_loss += loss.item() * len(y)
    return total_loss / len(loader.dataset)
```

- 두 함수는 배치를 모델에 그대로 넘길 뿐이라, 모델 안에서 무엇을 하든 상관없다
- 오늘은 이 두 함수를 **한 글자도 바꾸지 않고** CNN에 쓴다

PART 02

Convolution

2. 용어 정의

용어	정의	이 수업의 예
픽셀	사진을 이루는 가장 작은 칸, 밝기 숫자 하나	MNIST는 28 × 28 = 784칸
채널	같은 크기의 숫자 판 한 장, 흑백은 1장, 컬러는 R, G, B 3장	MNIST는 1채널
kernel	사진 위를 옮겨 다니며 곱하고 더하는 작은 숫자 판 (filter라고도 한다)	3 × 3
feature map	kernel 하나가 사진 전체를 훑어 만든 결과 판	첫 Conv2d에서 16장
stride	kernel을 한 번에 옮기는 칸 수	1
padding	사진 테두리에 덧붙이는 0의 줄 수	1
pooling	영역마다 대표값 하나만 남겨 크기를 줄이는 계산	2 × 2 영역의 최댓값
receptive field	출력 한 칸이 원래 사진에서 보는 범위	3 × 3에서 10 × 10까지

2. 입력 데이터 규격

기호	이름	이번 데이터	설명
N	배치 크기	64	한 번에 넣는 사진 수
C	채널 수	1	흑백 1, 컬러 3
H	세로 칸 수	28	픽셀 줄 수
W	가로 칸 수	28	한 줄의 픽셀 수

점검 `x.shape`가 `(64, 1, 28, 28)`이면 규격에 맞다

픽셀 값 규격

$$x = \frac{v}{255}, \qquad 0 \le v \le 255 \;\Rightarrow\; 0 \le x \le 1$$

v는 원래 픽셀 값(0~255)이고, x는 모델에 들어가는 값이다

주의 Conv2d에는 4차원 `(N, C, H, W)`를 넣는다, 펴서 `(64, 784)`로 넣으면 실행 오류가 난다 (Expected 3D or 4D input to conv2d)

2. Convolution 계산 절차

- **작업 순서**

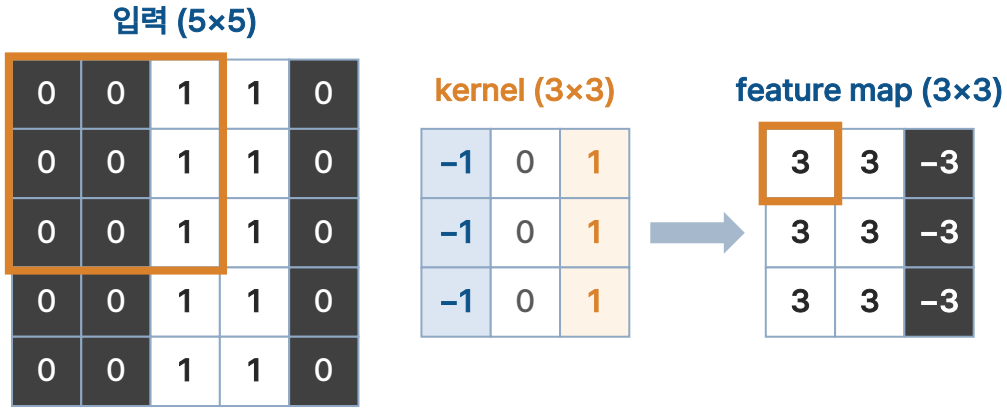
(1) kernel을 입력의 왼쪽 위 3 × 3 칸에 겹친다

(2) 겹친 칸끼리 숫자를 곱한다 (9번)

(3) 곱한 값 9개를 모두 더하고 bias를 더한다

(4) 결과를 feature map의 같은 자리에 적는다

(5) kernel을 stride만큼 옮겨 (1)~(4)를 되풀이한다



주황 칸은 줄마다 0 × (−1) + 0 × 0 + 1 × 1 = 1, 세 줄을 더하면 3이다

계산식

$$y_{i,j} = \sum_{m=0}^{2} \sum_{n=0}^{2} x_{i+m,\, j+n}\, w_{m,n} + b$$

참고 획의 왼쪽 경계는 +3, 오른쪽 경계는 −3이 된다, kernel의 숫자는 사람이 정하지 않고 학습으로 찾는다 (Conv2d)

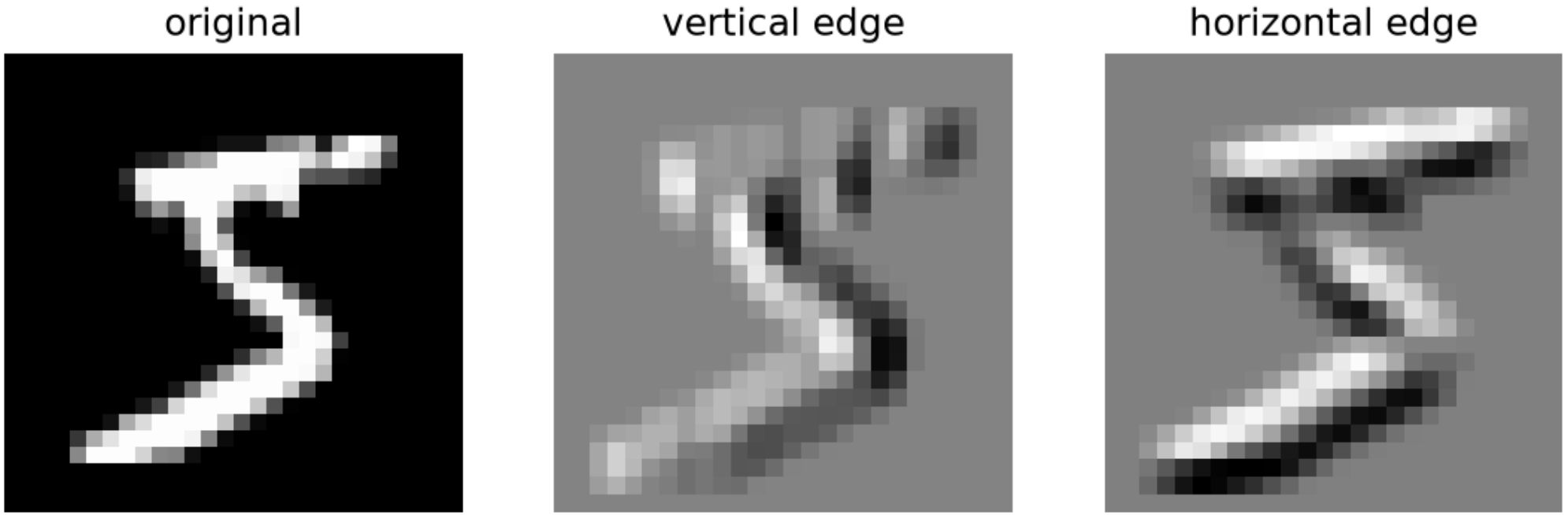
2. 예제 5-1: 손으로 만든 kernel 적용하기

예제 5-1. 세로 경계 kernel과 가로 경계 kernel

```
import torch.nn.functional as F
img, label = train_all[0]                # 첫 번째 학습 사진 (1, 28, 28)
x = img.view(1, 1, 28, 28)               # (N, C, H, W), 1장짜리 배치
k_v = torch.tensor([[-1., 0., 1.],
                    [-1., 0., 1.],
                    [-1., 0., 1.]])      # 세로 경계 kernel
k_h = torch.tensor([[-1., -1., -1.],
                    [ 0.,  0.,  0.],
                    [ 1.,  1.,  1.]])    # 가로 경계 kernel
out_v = F.conv2d(x, k_v.view(1, 1, 3, 3))   # kernel 모양도 4차원
out_h = F.conv2d(x, k_h.view(1, 1, 3, 3))
print(label, x.shape)      # → 5 torch.Size([1, 1, 28, 28])
print(out_v.shape)         # → torch.Size([1, 1, 26, 26])
```

- **읽는 법**
  - `F.conv2d`는 넘겨준 kernel로 **계산만** 하는 함수다, 여기서는 직접 정한 고정 kernel을 넘긴다
  - 입력은 (N, C, H, W) = (장수, 채널, 세로, 가로)이고, 흑백 사진 1장은 **(1, 1, 28, 28)**이다
  - kernel도 4차원으로 넣는다
- **결과**
  - 3×3 kernel은 가장자리 밖으로 못 나가 28×28이 **26×26**이 된다

2. 결과: kernel이 경계를 찾았다



예제 5-1 그리기 셀의 출력, 왼쪽부터 원본, 세로 경계 kernel, 가로 경계 kernel (회색이 0)

- 세로 경계 kernel은 세로 획의 **왼쪽 경계를 흰색**(양수), **오른쪽 경계를 검은색**(음수)으로 표시한다
- 밝기가 고른 곳은 0(회색)이다, kernel 숫자 9개가 **무엇을 찾을지** 정한다

2. 실습 1: kernel 숫자 바꿔 보기 (문제)

실습

1. 9칸 모두 1/9인 kernel을 쓰면 그림이 어떻게 될지 예측하고 확인하기
2. 가운데만 1이고 나머지가 0인 kernel이면?
3. 대각선 경계에 반응하는 kernel을 직접 만들어 보기

실습 1 셀. 첫 줄만 바꿔 가며 실행

```
k = torch.ones(3, 3) / 9              # 이 줄의 kernel만 바꿔 가며 실행한다
out = F.conv2d(x, k.view(1, 1, 3, 3))  # 예제 5-1의 사진 x에 적용
plt.imshow(out[0, 0], cmap="gray")
plt.axis("off")
plt.show()
```

- 먼저 결과 그림을 말로 예측하고 실행한다

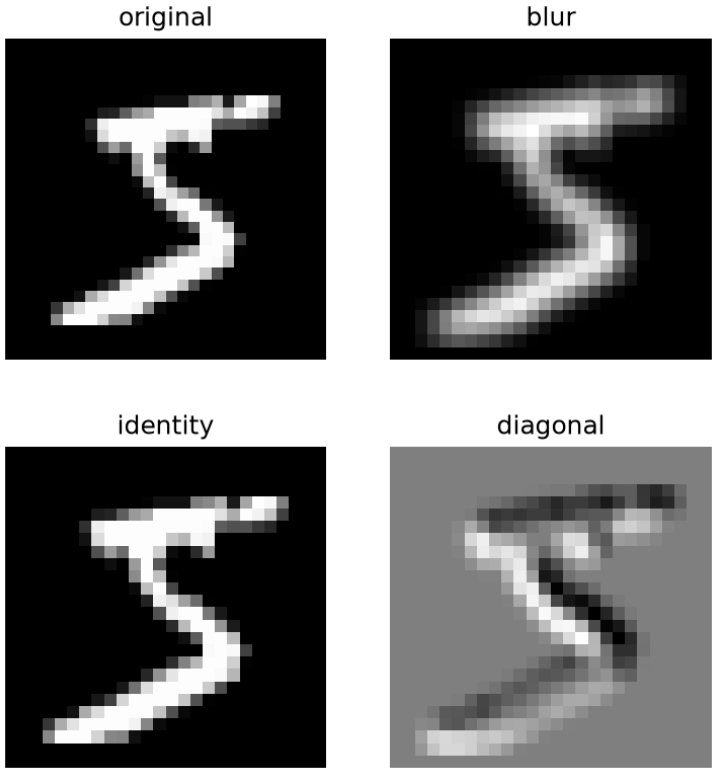
- **힌트**
  - 1은 이 셀을 그대로 실행
  - 2와 3은 첫 줄을 예제 5-1의 `k_v`처럼 `torch.tensor`로 3×3을 직접 쓰는 형태로
  - 3은 한쪽은 음수, 반대쪽은 양수로 두되 방향을 **대각선**으로
  - 예제 5-1의 셀은 **고치지 않는다**

2. 실습 1: 정답 예시

실습 1. 정답 예시, 실습 1 셀의 첫 줄을 셋 중 하나로 (한 번에 하나씩)

```
k = torch.ones(3, 3) / 9                # 1. 9칸 모두 1/9, 주변 평균
k = torch.tensor([[0., 0., 0.],
                  [0., 1., 0.],
                  [0., 0., 0.]])        # 2. 가운데만 1
k = torch.tensor([[ 0.,  1.,  1.],
                  [-1.,  0.,  1.],
                  [-1., -1.,  0.]])     # 3. 대각선 경계, 한 가지 예
```

- 1은 9칸의 평균이라 **흐려지고**, 2는 가운데만 1이라 가장자리 한 칸만 잘린 **원본 그대로**다
- 3은 한쪽은 양수, 반대쪽은 음수로 두면 **대각선 경계**에 반응한다
- 이 숫자를 **학습으로 찾게** 하는 것이 다음 장이다



세 kernel을 각각 적용한 결과

2. Conv2d 구성품 규격

항목	규격	쉬운 설명
기능	kernel로 사진을 훑어 feature map을 만든다	같은 무늬를 사진 전체에서 찾는다
입력	`(N, C_in, H, W)`	사진 N장, 채널 C_in장
출력	`(N, C_out, H_out, W_out)`	kernel 하나가 feature map 한 장을 만든다
설정값	`in_channels`, `out_channels`, `kernel_size`, `stride`, `padding`	입력 채널, kernel 개수, 크기, 옮기는 칸, 테두리
출력 크기	식 (1)	3 × 3, padding 1, stride 1이면 크기 그대로
파라미터 수	식 (2)	Conv2d(1, 16, 3)은 160개

식 (1) $$H_{\text{out}} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1$$

식 (2) $$N_{\text{param}} = C_{\text{out}} \times C_{\text{in}} \times K \times K + C_{\text{out}}$$

H는 입력 칸 수, K는 kernel 크기, P는 padding, S는 stride이고, ⌊ ⌋는 소수점을 버린다는 뜻이다

주의 `in_channels`가 입력의 채널 수와 다르면 실행 오류가 난다 (expected input to have 3 channels, but got 1 channels)

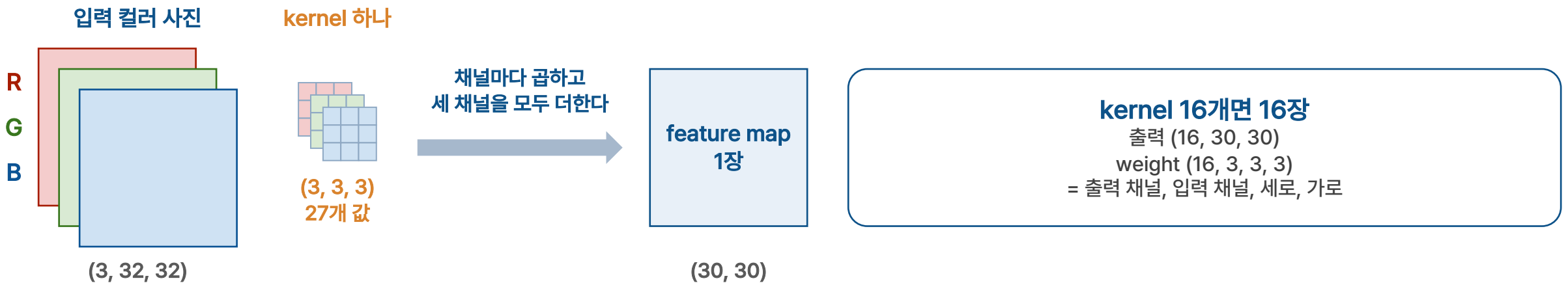
2. 예제 5-2: nn.Conv2d 만들어 보기

예제 5-2. 학습할 kernel 16개를 가진 계층

```
conv = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3)
print(conv.weight.shape)     # → torch.Size([16, 1, 3, 3])   (3×3 kernel 16개)
print(conv.bias.shape)       # → torch.Size([16])
print(sum(p.numel() for p in conv.parameters()))   # → 160   (16 × 9 + 16)
x, y = next(iter(train_loader))          # 배치 하나
print(x.shape)               # → torch.Size([64, 1, 28, 28])   (N, C, H, W)
print(conv(x).shape)         # → torch.Size([64, 16, 26, 26])   (feature map 16장씩)
```

- weight는 kernel 16개를 담은 **(16, 1, 3, 3)**이고, 각 kernel이 feature map 한 장을 만든다
- 배치 (64, 1, 28, 28)을 넣으면 **(64, 16, 26, 26)**, 사진마다 feature map 16장씩이다

2. 컬러 사진의 Convolution



식 (2)에 넣으면

$$N_{\text{param}} = 16\times(3\times3\times3) + 16 = 448$$

- kernel 하나도 입력 채널 수만큼 두꺼워(3 × 3 × 3), 세 채널을 모두 곱해 더해 feature map **한 장**을 만든다
- CNN의 conv2도 같다, 입력이 16장이면 kernel 하나가 16 × 3 × 3

2. 예제 5-3: 컬러 사진에 Conv2d 적용

예제 5-3. 컬러 사진 모양 (1, 3, 32, 32)에 적용

```
conv_rgb = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3)
print(conv_rgb.weight.shape)    # → torch.Size([16, 3, 3, 3])   (kernel 16개, 두께 3)
print(sum(p.numel() for p in conv_rgb.parameters()))   # → 448   (16 × 27 + 16)
photo = torch.rand(1, 3, 32, 32)          # 컬러 사진 1장 모양, 값은 무작위
print(conv_rgb(photo).shape)    # → torch.Size([1, 16, 30, 30])
```

- weight는 (출력 채널, 입력 채널, 세로, 가로) = **(16, 3, 3, 3)**이고, 파라미터는 16 × 27 + 16 = **448개**다
- 사진 크기와 상관없이 448개이고, 출력은 사진마다 feature map **16장**이다 (가장자리에서 줄어 30×30)

2. 예제 문제, Convolution 읽기

문제

1. 흑백 사진 묶음을 `(64, 28, 28)` 모양 그대로 `nn.Conv2d(1, 16, 3)`에 넣으면 어떻게 될까?
2. 사진이 28×28에서 280×280으로 커지면 `nn.Linear(입력, 100)`과 `nn.Conv2d(1, 16, 3)`의 파라미터 수는 각각 어떻게 변할까?
3. 세로 경계 kernel을 밝기 0.7로 고르게 칠한 사진에 적용하면 feature map은?

2. 예제 문제 풀이

2와 3을 실행해서 검증

```
print(sum(p.numel() for p in nn.Linear(784, 100).parameters()))
# → 78500   (문제 2, 28×28)
print(sum(p.numel() for p in nn.Linear(280 * 280, 100).parameters()))
# → 7840100   (문제 2, 280×280, 약 100배)
print(sum(p.numel() for p in nn.Conv2d(1, 16, 3).parameters()))
# → 160   (문제 2, 사진 크기와 상관없다)
flat = torch.full((1, 1, 5, 5), 0.7)   # 3. 고르게 0.7인 사진
print(F.conv2d(flat, k_v.view(1, 1, 3, 3)).view(-1).tolist())
# → [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
print(k_v.sum().item())                # → 0.0   (kernel 값의 합)
```

- **풀이**
  - 1. 오류가 난다, 채널 차원이 없으면 64를 **채널 수로 읽는다**
  - 2. Linear는 **약 100배**로 커지고 Conv2d는 **160개 그대로**다
  - 3. **전부 0**이다, kernel 값의 합이 0이라 밝기가 바뀌는 곳에서만 반응한다

1을 직접 실행하면

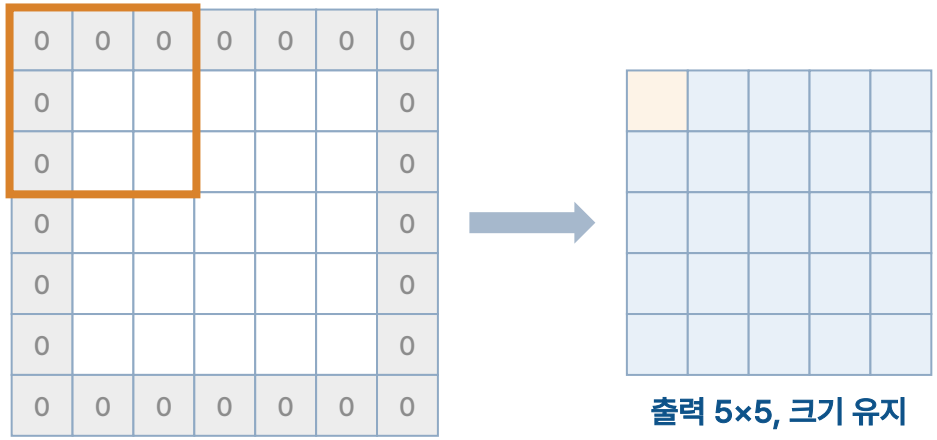
```
conv = nn.Conv2d(1, 16, kernel_size=3)
conv(torch.zeros(64, 28, 28))       # 1. 채널 차원 없이 넣으면
# → RuntimeError: ... expected input[1, 64, 28, 28] to have 1 channels, but got 64 channels instead
```

PART 03

feature map 크기 조절

3. 출력 크기 계산 절차

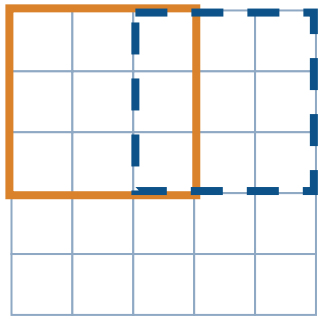
padding=1, 테두리를 0으로 한 칸 두른다



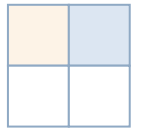
5×5 바깥에 0을 둘러 7×7

출력 5×5, 크기 유지

stride=2, kernel을 두 칸씩 옮긴다



첫 위치(주황), 다음 위치(파랑)



출력 2×2

식 (1) $$H_{\text{out}} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1$$

(1) 2P를 더한다
28 + 2 = 30

→ **(2) K를 뺀다**
30 − 3 = 27

→ **(3) S로 나누고 버린다**
27 ÷ 1 = 27

→ **(4) 1을 더한다**
27 + 1 = 28

참고 홀수 kernel K에 stride 1이면 padding (K − 1) ÷ 2로 크기가 그대로다, 3 × 3이면 padding 1

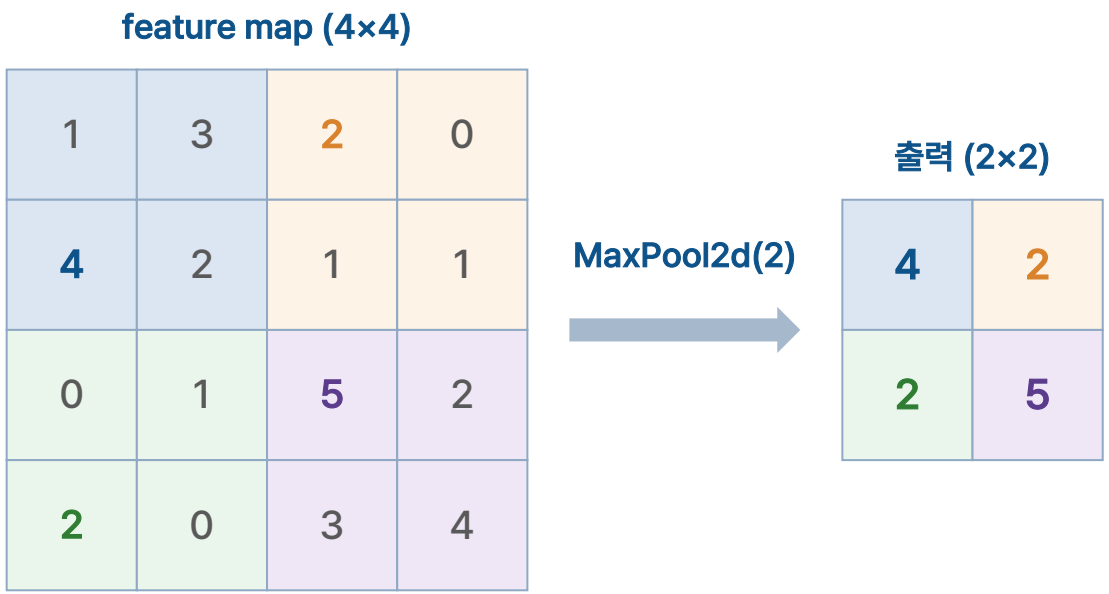
3. 예제 5-4: padding과 stride 확인

예제 5-4. 같은 사진, 다른 설정 세 가지

```
x = torch.zeros(1, 1, 28, 28)                                  # 사진 1장 모양
conv_a = nn.Conv2d(1, 8, kernel_size=3)                           # padding 없음
conv_b = nn.Conv2d(1, 8, kernel_size=3, padding=1)                # 테두리 1칸
conv_c = nn.Conv2d(1, 8, kernel_size=3, padding=1, stride=2)  # 2칸씩 이동
print(conv_a(x).shape)    # → torch.Size([1, 8, 26, 26])
print(conv_b(x).shape)    # → torch.Size([1, 8, 28, 28])    크기 유지
print(conv_c(x).shape)    # → torch.Size([1, 8, 14, 14])    절반
```

설정	식 (1)로 계산	출력 크기
kernel 3, padding 0, stride 1	(28 + 0 − 3) ÷ 1 + 1	26
kernel 3, padding 1, stride 1	(28 + 2 − 3) ÷ 1 + 1	**28**, 크기 유지
kernel 3, padding 1, stride 2	(28 + 2 − 3) ÷ 2 + 1, 13.5에서 버림	**14**, 절반

3. Pooling 구성품 규격



항목	규격
기능	영역마다 대표값 하나만 남긴다, MaxPool2d는 최댓값
입력	`(N, C, H, W)`
출력	`(N, C, H/2, W/2)`, 나눗셈은 버림, 채널 수는 그대로
설정값	`kernel_size=2`, stride를 주지 않으면 kernel_size와 같다
파라미터 수	0개, 학습하지 않는다
효과	계산량이 줄고 작은 위치 변화에 덜 민감해진다

참고 특징이 한 칸 정도 움직여도 같은 2 × 2 영역 안이면 결과가 같다, 그래서 작은 위치 변화에 덜 민감하다

3. 예제 5-5: MaxPool2d 확인

예제 5-5. 다이어그램과 같은 값으로 확인

```
pool = nn.MaxPool2d(2)                    # 2×2 영역마다 최댓값 하나
t = torch.tensor([[1., 3., 2., 0.],
                  [4., 2., 1., 1.],
                  [0., 1., 5., 2.],
                  [2., 0., 3., 4.]]).view(1, 1, 4, 4)
print(pool(t).view(2, 2).tolist())      # → [[4.0, 2.0], [2.0, 5.0]]
print(pool(torch.zeros(64, 16, 28, 28)).shape)
# → torch.Size([64, 16, 14, 14])   채널 수는 그대로, 크기만 절반
print(sum(p.numel() for p in pool.parameters()))   # → 0   파라미터 없음
```

- **결과**
  - 2×2 영역마다 최댓값 **4, 2, 2, 5**가 남는다
  - (64, 16, 28, 28)을 넣으면 채널은 그대로이고 크기만 **(64, 16, 14, 14)**로 줄어든다
  - 파라미터는 **0개**다

3. 예제 5-6: MaxPool2d와 AvgPool2d

예제 5-6. 예제 5-5의 t와 pool에 이어서

```
avg = nn.AvgPool2d(2)                          # 2×2 영역마다 평균 하나
print(pool(t).view(2, 2).tolist())             # → [[4.0, 2.0], [2.0, 5.0]]   (최댓값)
print(avg(t).view(2, 2).tolist())              # → [[2.5, 1.0], [0.75, 3.5]]   (평균)
print(sum(p.numel() for p in avg.parameters()))   # → 0
```

- MaxPool2d는 영역에서 **가장 강한 반응**만 남겨 특징이 있었는지를 보존한다
- AvgPool2d는 영역의 **평균**을 남겨 부드럽게 요약한다, 1, 3, 4, 2의 평균은 2.5다
- 중간 계층에는 MaxPool2d를 많이 쓰고, feature map 전체를 평균 내는 방식은 이후 ResNet에서 다시 본다

3. 실습 2: shape 예측하고 확인하기 (문제)

실습

1. 사진 1장 (1, 1, 28, 28)이 `nn.Conv2d(1, 8, kernel_size=5)`를 지나면 출력 shape는?
2. 1에 `padding=2`를 더하면?
3. 2에 `stride=2`를 더하면?

- **힌트**
  - 출력 크기 계산 절차의 식 (1)로 먼저 계산, 나눗셈은 소수점 버림
  - 확인은 예제 5-4처럼 `c1 = nn.Conv2d(1, 8, kernel_size=5)`로 계층을 만들고 `print(c1(x).shape)`로 한다
- **확인**
  - 예측을 종이에 먼저 적고 실행한다, 틀렸다면 어디서 틀렸는지 찾는다

3. 실습 2: 정답 예시

실습 2. 정답 예시, 주석이 계산 과정

```
x = torch.zeros(1, 1, 28, 28)
c1 = nn.Conv2d(1, 8, kernel_size=5)                        # 1
c2 = nn.Conv2d(1, 8, kernel_size=5, padding=2)             # 2
c3 = nn.Conv2d(1, 8, kernel_size=5, padding=2, stride=2)  # 3
print(c1(x).shape)    # → torch.Size([1, 8, 24, 24])   (28 − 5) ÷ 1 + 1
print(c2(x).shape)    # → torch.Size([1, 8, 28, 28])   (28 + 4 − 5) ÷ 1 + 1
print(c3(x).shape)    # → torch.Size([1, 8, 14, 14])   (28 + 4 − 5) ÷ 2 + 1, 버림
```

- 1. kernel 5는 가장자리 2칸씩 못 나가 **24×24**
- 2. padding=2로 두르면 **28×28** 그대로, 홀수 kernel K에 stride 1이면 padding (K − 1) ÷ 2로 크기 유지
- 3. 13.5에서 버린 13에 1을 더해 **14×14**

3. 예제 문제, 크기 계산 진단

문제

1. 28×28 feature map에 `MaxPool2d(2)`를 세 번 연속 적용하면 몇 × 몇이 되고, 7을 절반으로 나눌 때 무슨 일이 생길까?
2. padding 없는 3×3 Conv2d를 10번 연속 통과하면 사진 크기는 어떻게 되고, 무엇이 문제일까?
3. 크기를 절반으로 줄이는 `MaxPool2d(2)`와 `Conv2d(16, 16, 3, stride=2, padding=1)` 중 학습으로 바뀌는 파라미터가 있는 쪽은?

3. 예제 문제 풀이

실행해서 검증

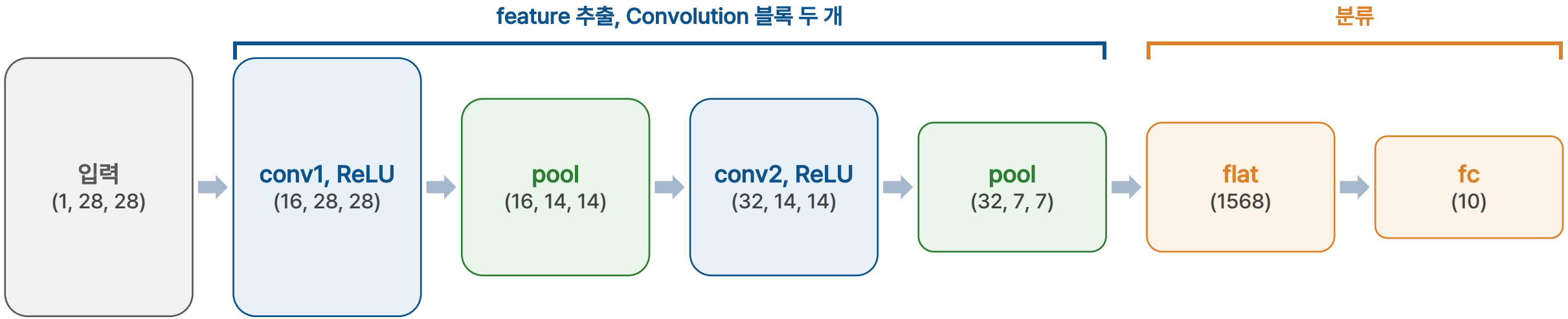
```
import torch
import torch.nn as nn
pool = nn.MaxPool2d(2)
x = torch.zeros(1, 1, 28, 28)
print(pool(pool(pool(x))).shape)   # → torch.Size([1, 1, 3, 3])   (문제 1)
conv = nn.Conv2d(1, 1, kernel_size=3)       # 2. padding 없는 3×3을
for i in range(10):                         #    10번 통과
    x = conv(x)
print(x.shape)                      # → torch.Size([1, 1, 8, 8])   (문제 2)
down = nn.Conv2d(16, 16, kernel_size=3, stride=2, padding=1)
print(sum(p.numel() for p in down.parameters()))   # → 2320   (문제 3)
print(sum(p.numel() for p in pool.parameters()))   # → 0
```

- **풀이**
  - 1. 28 → 14 → 7 → **3×3**, 7 ÷ 2 = 3.5에서 소수점을 버리므로 마지막 행과 열이 하나씩 빠진다
  - 2. 한 번에 2씩 줄어 **8×8**이 되고, 가장자리 픽셀은 가운데보다 적은 위치에만 반영된다, 깊은 CNN이 padding을 쓰는 이유다
  - 3. **Conv2d**다, 줄이는 방법을 파라미터로 학습하고, MaxPool2d는 파라미터 없이 고정된 규칙을 쓴다

PART 04

CNN 계통과 조립

4. CNN 계통도



- Conv, ReLU, MaxPool을 묶은 **블록 두 개**가 feature 추출(특징 추출)을 하고, 마지막 **flat과 fc**가 10개 점수를 낸다
- 블록을 지날 때마다 크기는 절반(28 → 14 → 7), 채널은 늘어난다(1 → 16 → 32), CNN에서 흔한 설계다

4. CNN 구성품 목록

번호	구성품	입력 → 출력 (사진 1장)	파라미터	곱셈 횟수
1	`Conv2d(1, 16, 3, padding=1)`	(1, 28, 28) → (16, 28, 28)	160	112,896
2	`ReLU`	모양 그대로	0	0
3	`MaxPool2d(2)`	→ (16, 14, 14)	0	0
4	`Conv2d(16, 32, 3, padding=1)`	→ (32, 14, 14)	4,640	903,168
5	`ReLU`	모양 그대로	0	0
6	`MaxPool2d(2)`	→ (32, 7, 7)	0	0
7	`Flatten`	→ 1,568	0	0
8	`Linear(1568, 10)`	→ 10	15,690	15,680
합계			**20,490**	**1,031,744**

점검 파라미터 합계가 20,490이고 출력이 `(N, 10)`이면 조립 정상이다

참고 MLP(파라미터 79,510개, 곱셈 79,400번)보다 파라미터는 약 4분의 1이고, 곱셈은 약 13배다

4. CNN의 발전

오늘 만들 CNN과 닮은 구조

1998, LeNet-5
손글씨 숫자(수표) 인식
conv와 pool 두 번, 이어서 fc
파라미터 약 6만 개

2012, AlexNet
ImageNet 대회 우승
오류율을 크게 낮춤
GPU 학습, ReLU, 8층

2014, VGG
3×3 conv를 깊게 쌓기
16~19층
오늘 배운 설정을 반복

2015, ResNet
skip connection으로
152층까지 쌓기
ResNet-18도 이 계열



더 깊게, 더 큰 데이터(ImageNet)로, 기본 블록은 Conv, ReLU, Pool

- 오늘 만들 CNN은 1998년 LeNet-5처럼 Conv와 Pool을 두 번 거친 뒤 fc로 분류하는 구조다
- 이후 CNN은 GPU와 큰 데이터를 만나 깊어졌지만, 기본 블록은 오늘 배운 것과 같다

4. 예제 5-7: CNN 클래스

예제 5-7. MLP와 같은 nn.Module 틀

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.act = nn.ReLU()
        self.flat = nn.Flatten()
        self.fc = nn.Linear(32 * 7 * 7, 10)
    def forward(self, x):
        x = self.pool(self.act(self.conv1(x)))   # (N, 16, 14, 14)
        x = self.pool(self.act(self.conv2(x)))   # (N, 32, 7, 7)
        return self.fc(self.flat(x))             # (N, 1568) → (N, 10)

print(CNN()(torch.zeros(2, 1, 28, 28)).shape)    # 사진 2장 모양으로 확인
# → torch.Size([2, 10])
```

- **읽는 법**
  - `__init__`에서 계층을 만들고 `forward`에서 순서대로 통과시키는 틀은 그대로다
  - pool과 act는 파라미터가 없어 **하나를 두 번** 써도 된다
- **숫자를 맞춰야 하는 곳**
  - conv1의 출력 채널(16)과 conv2의 입력 채널(16)이 같아야 한다
  - fc의 입력(32 × 7 × 7 = **1568**)이 마지막 feature map의 원소 수와 같아야 한다

4. 예제 5-8: shape 따라가기

예제 5-8. 배치 하나 모양을 계층별로 통과시키기

```
model = CNN()
x = torch.zeros(64, 1, 28, 28)                    # 배치 하나 모양
h1 = model.pool(model.act(model.conv1(x)))        # 첫 번째 블록
h2 = model.pool(model.act(model.conv2(h1)))       # 두 번째 블록
print(model.conv1(x).shape)   # → torch.Size([64, 16, 28, 28])   (padding=1, 크기 유지)
print(h1.shape)               # → torch.Size([64, 16, 14, 14])   (pool로 절반)
print(h2.shape)               # → torch.Size([64, 32, 7, 7])
print(model.flat(h2).shape)   # → torch.Size([64, 1568])   (32 × 7 × 7)
print(model(x).shape)         # → torch.Size([64, 10])
```

- conv는 **채널**을 바꾸고 pool은 **크기**를 절반으로 줄인다, flat이 (32, 7, 7)을 **1568**개로 편다
- 새 모델을 만들면 이렇게 한 계층씩 shape를 찍어 보는 것이 가장 확실한 점검법이다

4. 예제 5-9: 파라미터 세어 보기

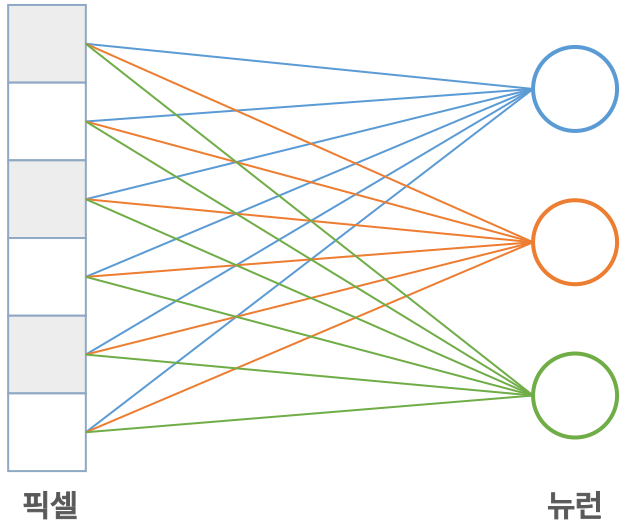
예제 5-9. 계층별 파라미터 수

```
model = CNN()
for name, p in model.named_parameters():   # 이름과 함께 꺼낸다
    print(name, list(p.shape), p.numel())
# → conv1.weight [16, 1, 3, 3] 144
# → conv1.bias [16] 16
# → conv2.weight [32, 16, 3, 3] 4608
# → conv2.bias [32] 32
# → fc.weight [10, 1568] 15680
# → fc.bias [10] 10
print(sum(p.numel() for p in model.parameters()))   # → 20490
print(sum(p.numel() for p in MLP().parameters()))   # → 79510
```

- **읽는 법**
  - conv2의 kernel 하나는 입력 16장의 같은 자리를 **모두 곱해 더해** 출력 한 칸을 만들므로 크기가 16 × 3 × 3이다
- **결과**
  - Convolution 두 계층은 bias까지 **4,800개**뿐이고, 가장 많은 것은 마지막 fc의 15,690개다
  - 전체 **20,490개**로 MLP 79,510개의 약 4분의 1이다

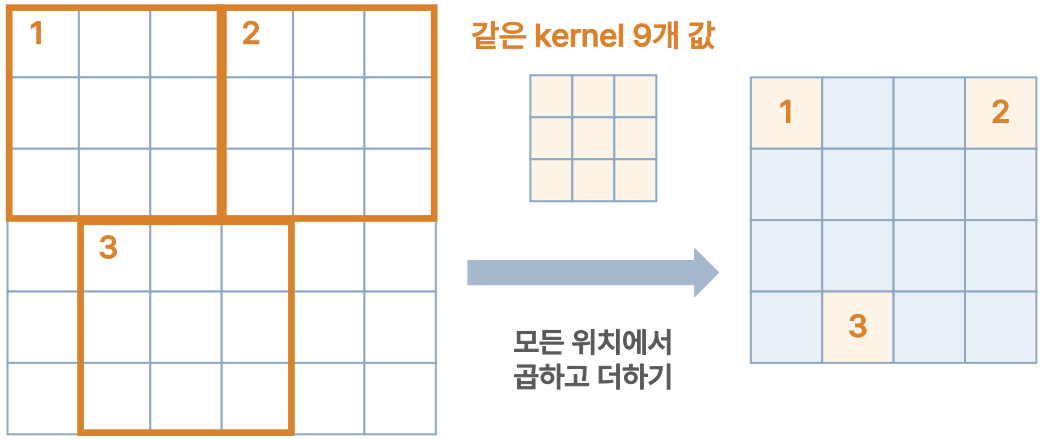
4. weight sharing

MLP의 fc1, 연결마다 weight가 따로



weight 78,400개
픽셀 784 × 뉴런 100

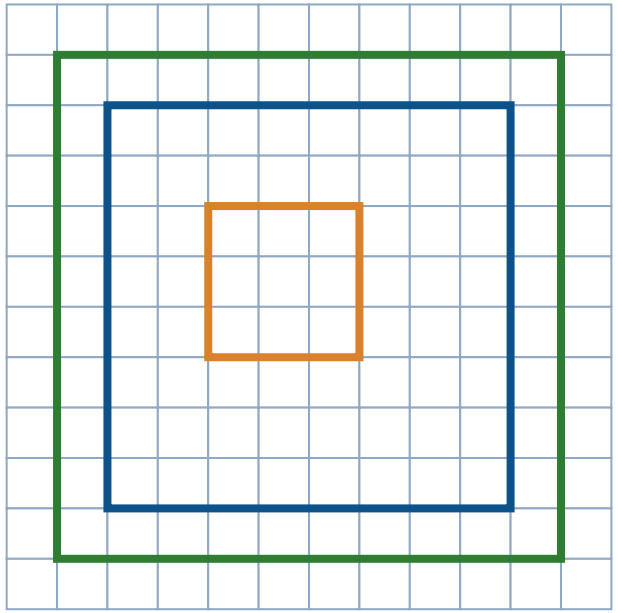
conv1, 같은 kernel을 모든 위치에



weight 144개
kernel 16개 × 9, 사진 크기와 상관없다

- Conv2d는 같은 kernel을 모든 위치에 다시 쓰므로(**weight sharing**) 파라미터가 사진 크기와 상관없다

4. receptive field



conv1 출력 한 칸
원래 사진의 3×3을 본다

pool 뒤 conv2 출력 한 칸
원래 사진의 8×8을 본다

마지막 feature map(7×7) 한 칸
원래 사진의 10×10을 요약한다

- **receptive field(수용 영역)**는 출력 한 칸이 원래 사진에서 보는 영역이다
- 3×3에서 pool 뒤 4×4, conv2 뒤 8×8, 다시 pool 뒤 **10×10**으로 넓어진다, kernel은 계속 3×3인데 원래 사진 기준으로 더 넓게 본다

4. 예제 문제, CNN 코드 진단

문제

1. fc를 `nn.Linear(32 * 14 * 14, 10)`으로 잘못 쓰면 언제, 어떤 오류가 날까?
2. conv1의 출력 채널만 16에서 32로 바꾸고 나머지는 그대로 두면?
3. 같은 CNN에 32×32 크기의 사진을 넣으면?

4. 고장 탐구: CNN 코드 오류

번호	증상 (오류 메시지)	원인	조치
1	만들 때는 조용하다가 첫 forward의 fc에서 `(64x1568 and 6272x10)`	fc 입력을 32 × 14 × 14로 잘못 썼다	fc 입력을 32 × 7 × 7로
2	conv2에서 `expected ... 16 channels, but got 32`	conv1의 출력 채널만 32로 바꿨다	conv2의 입력 채널도 32로
3	conv와 pool은 통과하고 fc에서 `(64x2048 and 1568x10)`	32×32 사진이라 마지막 feature map이 8 × 8이다	fc 입력을 32 × 8 × 8로, 또는 사진을 28×28로

1을 직접 실행하면

```
x = torch.zeros(64, 1, 28, 28)
m = CNN()
m.fc = nn.Linear(32 * 14 * 14, 10)    # 1. fc 크기를 잘못 쓰면, 만들 때는 조용하다
m(x)
# → RuntimeError: mat1 and mat2 shapes cannot be multiplied (64x1568 and 6272x10)
```

참고 오류 메시지의 숫자에 답이 있다, 1568은 32 × 7 × 7, 6272는 32 × 14 × 14다

4. 실습 3: 블록 하나 더 쌓기 (문제)

실습

1. 실습 3 셀에 conv3(32 → 64채널, 3×3, padding 1)과 pool 블록을 넣어 CNN3 완성하기
2. (2, 1, 28, 28)을 넣어 (2, 10)이 나오도록 fc의 입력 숫자 맞추기
3. 파라미터 수를 세어 CNN(20,490개)과 비교하고, 7×7이 pool을 지나면 몇 × 몇이 되는지 설명하기

- **힌트**
  - 노트북의 실습 3 셀은 예제 5-7을 복사해 이름만 CNN3으로 바꾼 틀, '고칠 곳' 주석 1~3만 고친다
  - 마지막 feature map 크기를 식 (1)로 먼저 계산, 7 ÷ 2는 소수점 버림
  - 오류가 나면 고장 탐구표처럼 오류 메시지의 숫자부터 읽는다
- **확인**
  - `torch.Size([2, 10])`과 파라미터 수 **29066**이 함께 나오고, forward에 conv3 블록이 들어갔으면 성공

4. 실습 3: 정답 예시

실습 3. 정답 예시, 고칠 곳 1~3

```
class CNN3(nn.Module):                          # 예제 5-7을 복사해 블록 하나 추가
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)   # 고칠 곳 1
        self.pool = nn.MaxPool2d(2)
        self.act = nn.ReLU()
        self.flat = nn.Flatten()
        self.fc = nn.Linear(64 * 3 * 3, 10)                        # 고칠 곳 2
    def forward(self, x):
        x = self.pool(self.act(self.conv1(x)))   # (N, 16, 14, 14)
        x = self.pool(self.act(self.conv2(x)))   # (N, 32, 7, 7)
        x = self.pool(self.act(self.conv3(x)))   # (N, 64, 3, 3)   고칠 곳 3
        return self.fc(self.flat(x))

print(CNN3()(torch.zeros(2, 1, 28, 28)).shape)       # → torch.Size([2, 10])
print(sum(p.numel() for p in CNN3().parameters()))    # → 29066
```

- **포인트**
  - 7 ÷ 2 = 3.5에서 버려 마지막 feature map은 **(64, 3, 3)**이고, fc 입력은 **576**(64 × 3 × 3)이다
  - conv3이 18,496개 늘고 fc가 9,920개 줄어 전체 **29,066**개다
- **학습해 보면 (참고 실측)**
  - 같은 조건으로 best val **98.80%**, CNN보다 +0.26%p였다 (seed 한 번)

PART 05

CNN 학습과 평가

5. 예제 5-10: 같은 학습 코드로 CNN 학습

예제 5-10. Baseline 학습 코드에서 모델과 파일 이름 두 곳 변경, 출력 한 줄 추가

```
torch.manual_seed(0)                          # Baseline과 같은 seed
model = CNN().to(device)                      # 바꾼 곳 1, 모델
opt = torch.optim.SGD(model.parameters(), lr=0.1)
best_acc, best_epoch = 0.0, 0
for epoch in range(10):
    train_one_epoch(model, train_loader, opt)
    vl, va = evaluate(model, val_loader)
    print(epoch + 1, round(va, 2))            # 추가, 진행 상황 보기
    if va > best_acc:
        best_acc, best_epoch = va, epoch + 1
        torch.save(model.state_dict(), "exp2_cnn.pt")   # 바꾼 곳 2
print(best_epoch, round(best_acc, 2))         # → (예) 9 98.54
```

epoch	val 정확도
1	96.04%
2	92.79%
3	97.84%
4	97.05%
5	98.09%
6	98.23%
7	98.20%
8	98.40%
9	98.54%
10	98.38%

- 학습 함수와 검증 함수는 **그대로**이고 바꾼 것은 모델과 파일 이름이다
- best는 **9 epoch의 98.54%**다

5. 예제 5-11: 테스트 세트로 최종 평가

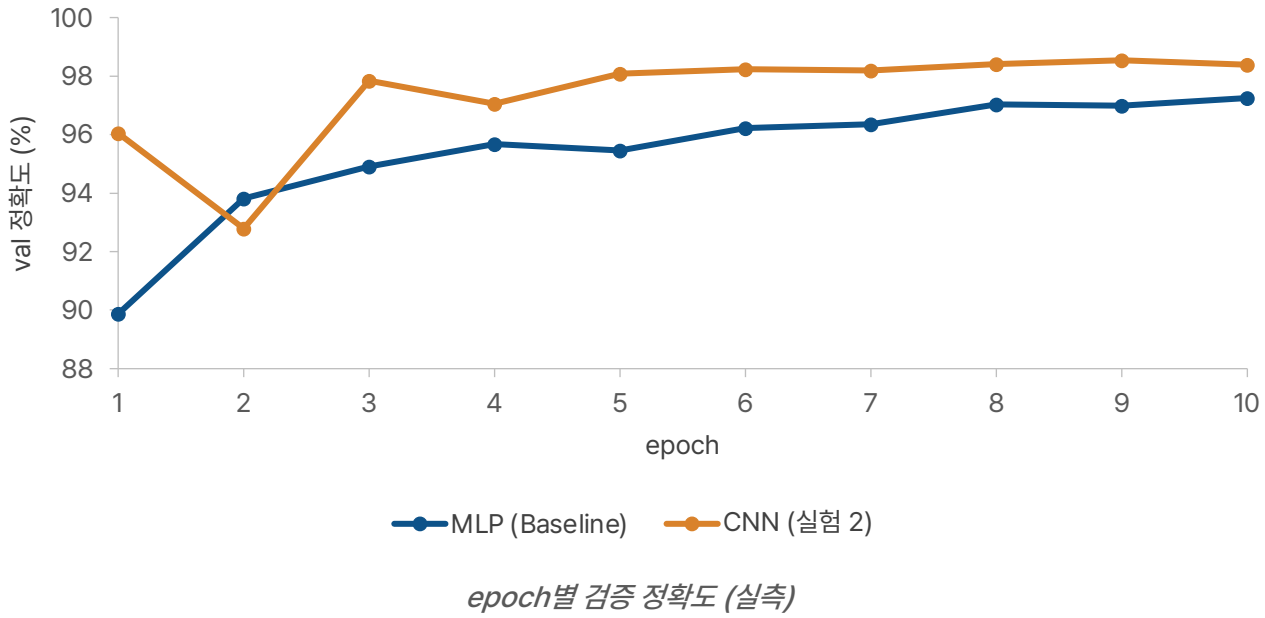
예제 5-11. Baseline 테스트 코드에서 모델과 파일 이름만 교체

```
test_set = datasets.MNIST(root="data", train=False, download=True,
                          transform=transforms.ToTensor())
test_loader = DataLoader(test_set, batch_size=256, shuffle=False)
model = CNN().to(device)
state = torch.load("exp2_cnn.pt", map_location=device)   # best 파라미터
model.load_state_dict(state)
test_loss, test_acc = evaluate(model, test_loader)
print(len(test_set), round(test_acc, 2))      # → (예) 10000 98.67
```

- test 정확도는 **98.67%**다, Baseline의 97.34%보다 +1.33%p이고 기록용으로 남긴다
- 판단은 **검증 정확도**로

5. Baseline과 비교

실험	바꾼 것	best val (epoch)	test	Baseline 대비 (val)
Baseline	없음, MLP 784-100-10	97.25% (10)	97.34%	기준
실험 1	optimizer 설정, SGD lr 0.1 → Adam lr 0.001	97.43% (10)	97.59%	+0.18%p
실험 2	모델, MLP → CNN (파라미터 20,490개)	98.54% (9)	98.67%	**+1.29%p**



epoch별 검증 정확도 (실측)

- 파라미터는 약 4분의 1인데 검증 정확도는 **+1.29%p** 다
- CNN은 1 epoch 만에 96.04%로 MLP의 6 epoch 수준이고, 2 epoch처럼 흔들리는 구간도 있다
- 이 차이는 의미 있는 차이일까?

5. 예제 문제, 결과 해석

문제

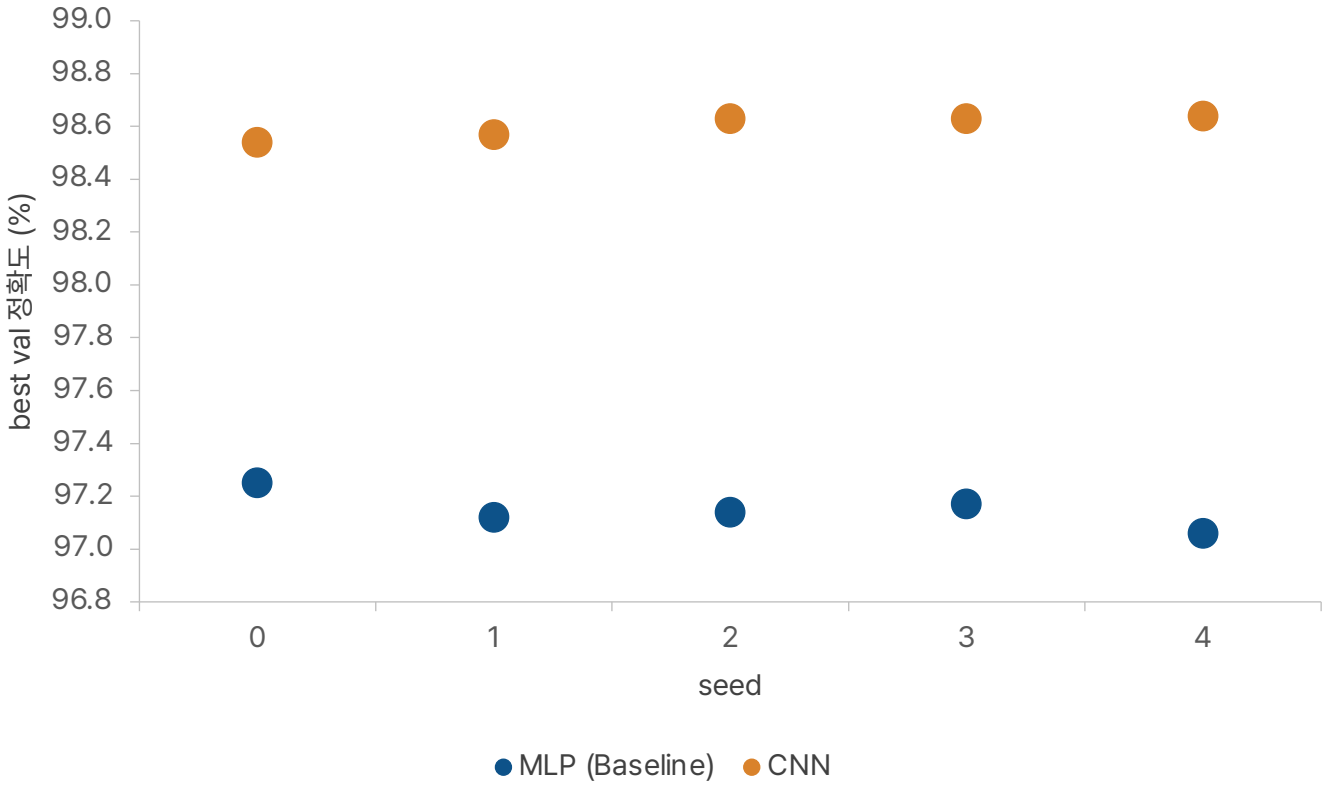
1. CNN의 검증 정확도가 2 epoch에 92.79%로 1 epoch(96.04%)보다 떨어졌다, 학습이 잘못된 걸까?
2. seed만 바꿔도 Baseline이 0.19%p 흔들렸다, CNN의 +1.29%p는 의미 있는 차이일까?
3. CNN의 파라미터는 MLP의 약 4분의 1이다, 그렇다면 계산량도 4분의 1일까?

5. 예제 문제 풀이

문제	풀이	근거 (실측)
1	아니다, lr 0.1의 SGD는 epoch마다 오르내릴 수 있다	3 epoch에 97.84%로 회복, 그래서 best를 저장한다
2	의미 있다	seed 다섯 번 모두 CNN이 **1%p 넘게** 높았다 (다음 장)
3	아니다, 계산은 오히려 **약 13배** 많다	사진 한 장의 곱셈 약 103만 번 대 8만 번, CPU에서 재면 약 6.9배 느리다

- 3. kernel을 모든 위치에서 다시 쓰므로 weight 하나가 여러 번 곱해진다, **파라미터 수와 계산량은 다르다**
- GPU에서는 epoch당 MLP 약 5초, CNN 약 6초로 차이가 작다, 데이터를 준비하는 시간이 대부분이기 때문이다

5. seed를 바꿔 다섯 번 돌려 보면



seed 0~4로 다섯 번씩 돌린 실측, 세로축 96.8~99.0%로 확대

- MLP는 **97.06~97.25%**, CNN은 **98.54~98.64%**로 다섯 번 모두 CNN이 높았다
- 가장 낮은 CNN도 가장 높은 MLP보다 1.29%p 높다, seed 변동보다 **훨씬 큰** 차이다
- 반복해도 같은 결론이니 이 조건에서는 **CNN이 낫다**고 말할 수 있다

5. 실습 4: CNN이 틀린 사진 구경하기 (문제)

실습

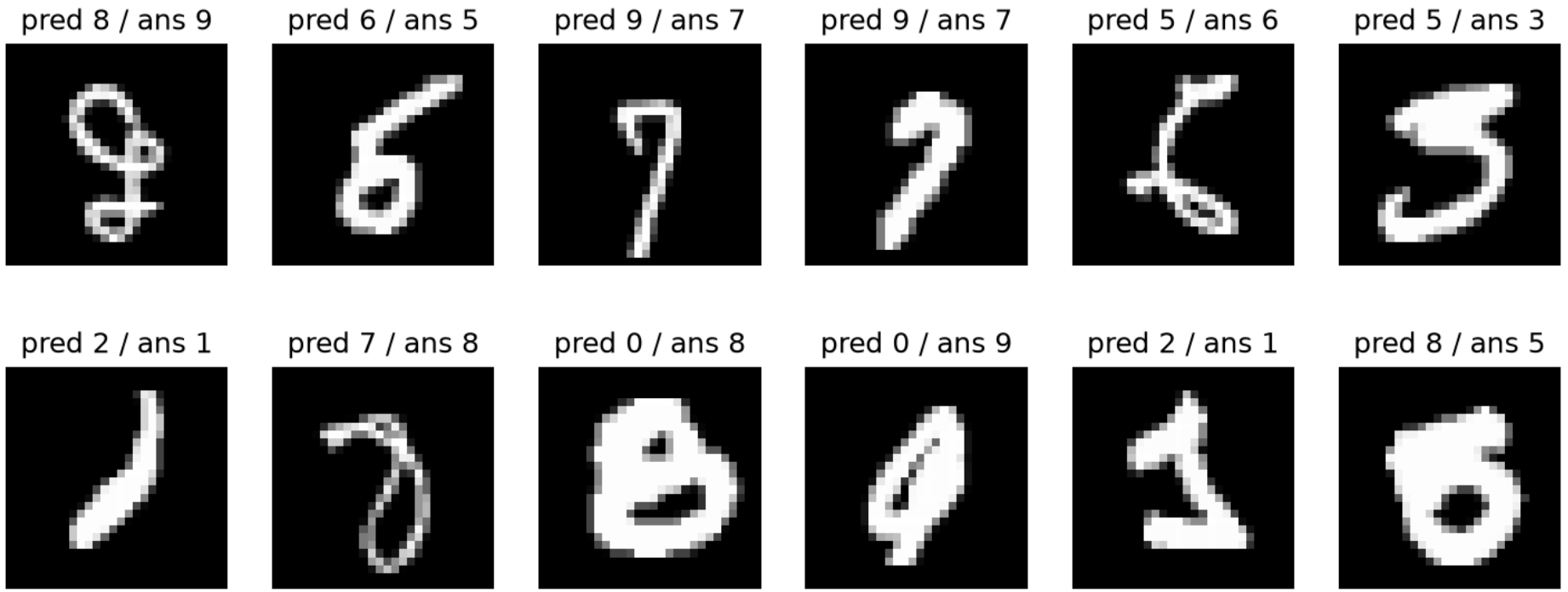
1. 실습 4 셀을 실행해 CNN이 틀린 검증 사진이 몇 장인지 확인하기
2. 그리기 셀로 틀린 사진 12장을 보고, 사람이 봐도 헷갈리는 사진이 몇 장인지 세어 보기
3. 가장 자주 헷갈린 (정답, 예측) 숫자 쌍을 찾고 이유를 생각해 보기

실습 4 셀. 예제 5-11의 best CNN으로 검증 사진 10,000장 예측

```
model.eval()                                    # 예제 5-11에서 불러온 best CNN
preds, labels = [], []
with torch.no_grad():
    for x, y in val_loader:                     # 검증 데이터 10,000장
        preds.append(model(x.to(device)).argmax(dim=1).cpu())   # 배치마다 예측
        labels.append(y)                        # 배치마다 정답
preds, labels = torch.cat(preds), torch.cat(labels)   # 10,000개씩 한 줄로
wrong = (preds != labels).nonzero().view(-1)    # 틀린 사진 번호들
print(len(wrong))                               # → (예) 146
print(wrong[:5].tolist())                       # → (예) [2, 35, 172, 204, 217]
```

- 개선 방향을 찾는 분석이라 테스트가 아니라 **검증 데이터**로, 테스트로 정하면 테스트가 검증이 되어 버린다

5. 실습 4: 정답 예시



그리기 셀의 출력, pred는 CNN의 예측, ans는 정답

- CNN은 검증 사진 10,000장 중 **146장**을 틀렸고, MLP Baseline은 275장을 틀렸다
- 둘 다 틀린 사진은 93장이다, 사람이 봐도 헷갈리는 글씨가 많다

5. 결과: 자주 헷갈린 숫자 쌍

실습 4의 셋째. (정답, 예측) 쌍 세기

```
from collections import Counter
pairs = Counter(zip(labels[wrong].tolist(), preds[wrong].tolist()))
for pair, n in pairs.most_common(6):     # (정답, 예측) 쌍, 많은 순서로 6개
    print(pair, n)
# → (예) (7, 9) 9
# → (예) (3, 8) 9
# → (예) (1, 2) 8
# → (예) (3, 5) 7
# → (예) (4, 9) 7
# → (예) (3, 2) 6
```

정답 → 예측	횟수
7 → 9	9번
3 → 8	9번
1 → 2	8번
3 → 5	7번
4 → 9	7번
3 → 2	6번

- 7과 9, 3과 8, 3과 5처럼 **획 모양이 닮은** 쌍이 많다
- 어떤 쌍에서 틀리는지 알면 데이터를 어디서 보강할지 보인다, 이후 데이터 증강과 오분류 분석에서 다시 쓴다

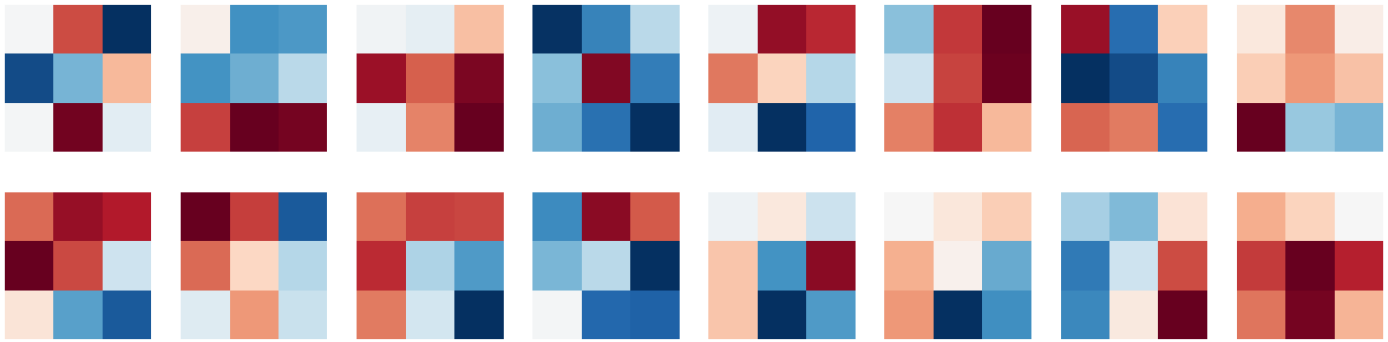
PART 06

CNN이 강한 이유

6. 예제 5-12: 학습된 kernel 16개

예제 5-12. 예제 5-11의 best CNN에서 conv1의 kernel

```
import matplotlib.pyplot as plt
w = model.conv1.weight.detach().cpu()    # 학습된 kernel 16개 (16, 1, 3, 3)
fig, axes = plt.subplots(2, 8, figsize=(12, 3))
for i, ax in enumerate(axes.flat):       # 칸 16개를 차례로
    v = w[i, 0].abs().max()              # 0이 흰색이 되도록 색 범위를 ±v로
    ax.imshow(w[i, 0], cmap="RdBu_r", vmin=-v, vmax=v)
    ax.axis("off")
plt.show()
```



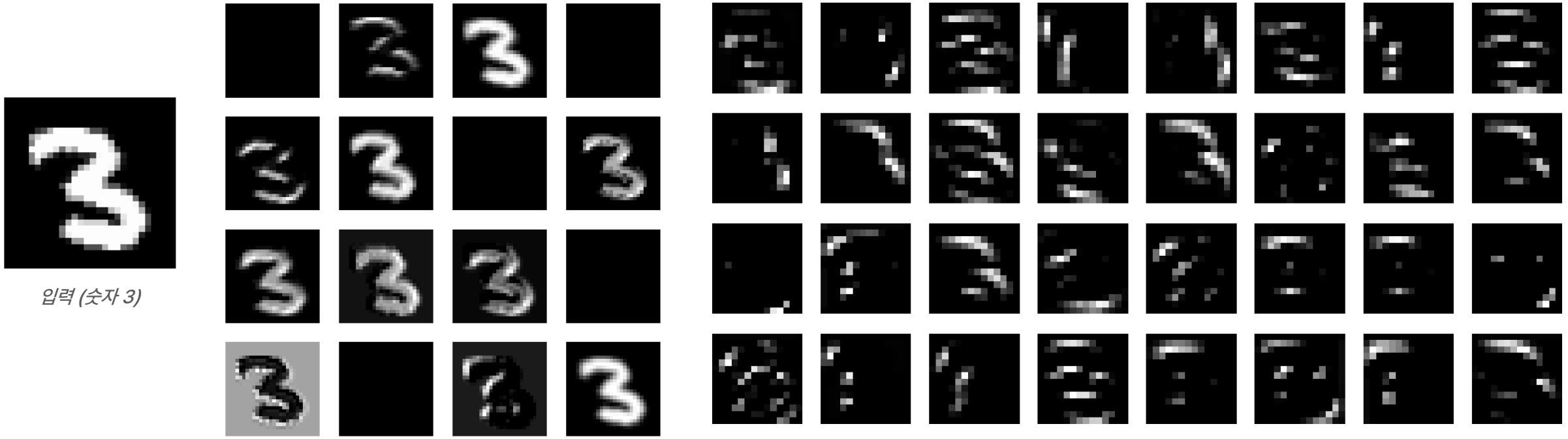
학습된 conv1 kernel 16개 (그림마다 색 범위가 다르고 0이 흰색)

- 빨강은 양수, 파랑은 음수, 흰색은 0
- 한쪽은 양수, 반대쪽은 음수인 **경계 kernel과 닮은 것**이 학습 전 3개에서 학습 후 7개로 늘었다
- 사람이 정하지 않았는데 학습이 kernel을 **경계를 찾는 쪽**으로 바꿨다

6. 예제 5-13: 숫자 한 장의 feature map

첫 번째 계층, feature map 16장 (28×28)

두 번째 계층, feature map 32장 (14×14)



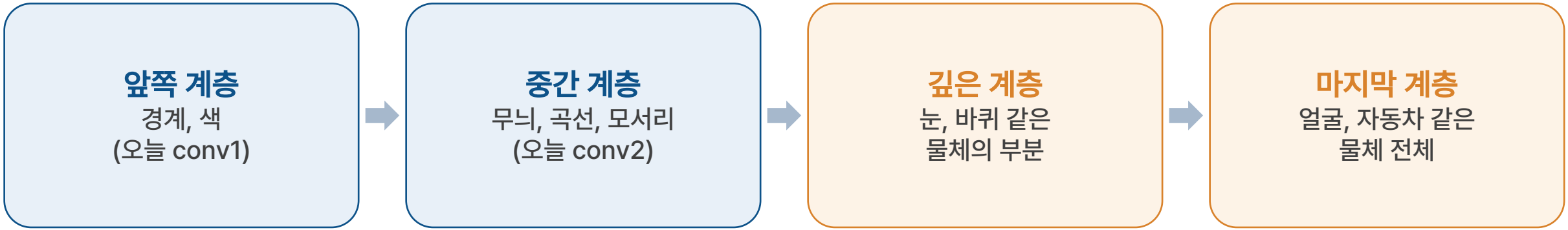
입력 (숫자 3)

16장을 4×4로 배치

- 첫 번째 계층은 획 전체나 위, 아래 경계처럼 **단순한 모양**에 반응한다, 검은 칸은 이 사진에서 값이 모두 0인 feature map이다
- 두 번째 계층은 크기가 절반인 대신 가로 획, 오른쪽 곡선처럼 **특정 부분**에 반응하는 것이 많다

6. 특징의 계층

ImageNet으로 학습한 큰 CNN에서 관찰된 경향



receptive field가 넓어지며 앞 계층의 특징을 조합한다

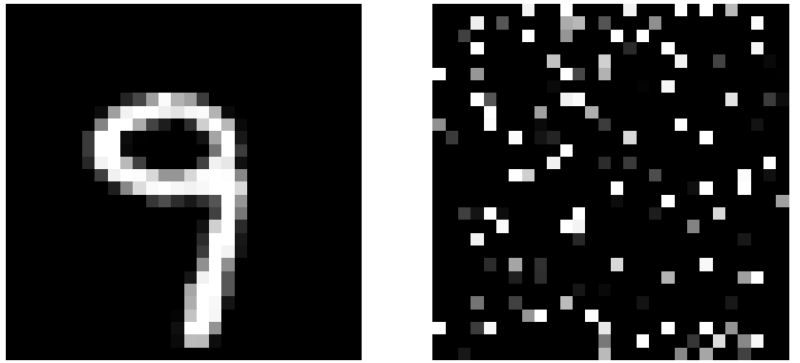
- 오늘 CNN(블록 두 개)은 앞의 두 단계, 경계와 획의 부분까지 보여 준다
- 이후 배울 전이학습은 큰 CNN이 배워 둔 특징 추출 계층을 다른 문제에 다시 쓰는 방법이다

6. 예제 5-14: 픽셀을 섞은 데이터 만들기

예제 5-14. 모든 사진을 같은 규칙으로 섞는 transform

```
perm = torch.randperm(784, generator=torch.Generator().manual_seed(123))
def shuffle_pixels(img):                    # 모든 사진을 같은 규칙으로 섞는다
    return img.view(-1)[perm].view(1, 28, 28)
tf = transforms.Compose([transforms.ToTensor(), shuffle_pixels])
train_all_s = datasets.MNIST(root="data", train=True, download=True,
                             transform=tf)
g = torch.Generator().manual_seed(0)        # 같은 분리
train_s, val_s = random_split(train_all_s, [50000, 10000], generator=g)
train_loader_s = DataLoader(train_s, batch_size=64, shuffle=True)
val_loader_s = DataLoader(val_s, batch_size=256, shuffle=False)
print(train_set[0][1], train_s[0][1])       # → (예) 9 9   같은 사진, 같은 라벨
```

- `transforms.Compose`는 전처리를 순서대로 묶는다, 사진을 받아 사진을 돌려주는 함수면 직접 만든 것도 넣을 수 있다



그리기 셀의 출력, 왼쪽 원래, 오른쪽 섞음

- `perm`은 784칸을 섞는 **순서 하나**다, 모든 사진에 같은 순서를 쓴다
- 픽셀 값과 라벨은 그대로이고 **자리만** 바뀌었다

6. 실습 5: 섞인 사진으로 다시 학습 (문제)

실습

1. 섞인 사진에서 MLP와 CNN의 best val이 각각 어떻게 될지 예측해 적기
2. 예제 5-10을 복사해 모델을 MLP로, 로더 두 곳을 `train_loader_s`와 `val_loader_s`로, 파일 이름을 바꿔 학습하고, 이어서 CNN도 학습하기
3. 원래 사진과 비교해 두 모델이 왜 다르게 변했는지 한 줄로 쓰기

모델	내 예측	원래 사진 best val	섞은 사진 best val	차이
MLP		97.25%		
CNN		98.54%		

- 파일 이름은 "shuffled_mlp.pt"와 "shuffled_cnn.pt"처럼 바꿔 **앞의 파일을 덮어쓰지 않는다**
- 학습은 각각 1분 남짓, 기다리는 동안 셋째를 생각한다

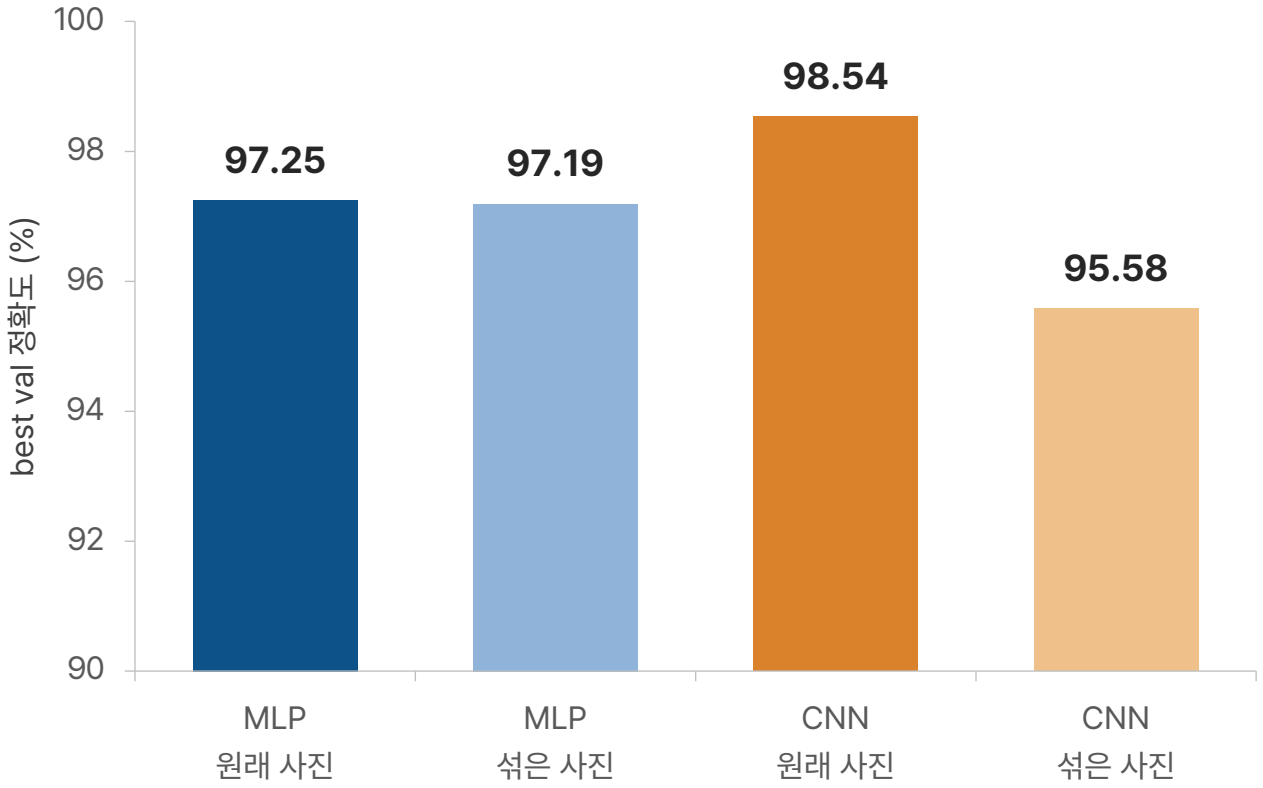
6. 실습 5: 정답 예시

실습 5. 정답 예시, MLP (CNN은 모델 줄과 파일 이름만 다시 바꿈)

```
torch.manual_seed(0)
model = MLP().to(device)                          # 바꾼 곳 1, 모델
opt = torch.optim.SGD(model.parameters(), lr=0.1)
best_acc, best_epoch = 0.0, 0
for epoch in range(10):
    train_one_epoch(model, train_loader_s, opt)   # 바꾼 곳 2
    vl, va = evaluate(model, val_loader_s)        # 바꾼 곳 3
    print(epoch + 1, round(va, 2))
    if va > best_acc:
        best_acc, best_epoch = va, epoch + 1
        torch.save(model.state_dict(), "shuffled_mlp.pt")   # 바꾼 곳 4
print(best_epoch, round(best_acc, 2))             # → (예) 10 97.19
```

- **결과 (실측)**
  - MLP는 **97.19%**로 원래 97.25%와 거의 같다 (-0.06%p)
  - CNN은 **95.58%**로 원래 98.54%에서 2.96%p 떨어져 **MLP보다 낮아졌다**
- **CNN 학습**
  - 이 코드에서 모델 줄을 `CNN()`으로, 파일 이름을 "shuffled_cnn.pt"로 바꾼다

6. 결과: CNN은 이웃 관계를 이용한다



같은 학습 조건, 같은 seed 0의 실측, 세로축은 90%부터

- MLP는 섞어도 거의 그대로다(-0.06%p), 이웃 관계를 **구조로 가정하지 않아** 섞인 순서에도 똑같이 적응하기 때문이다
- CNN은 2.96%p 떨어졌다, kernel이 보는 3×3 이웃 안에서 **경계와 획 모양이 사라졌기** 때문이다
- CNN은 **가까운 픽셀끼리 관련이 있다**는 사진의 성질을 구조에 넣은 모델이다

6. 실습 6: 사진을 옮기면? (문제)

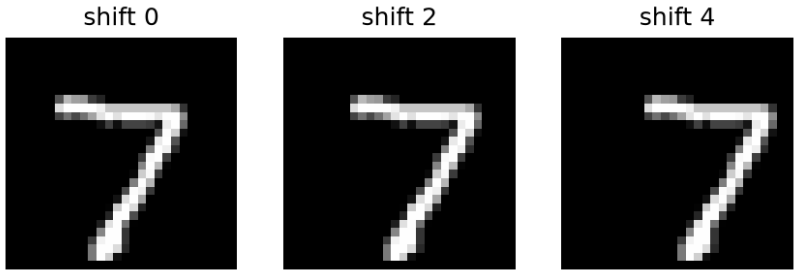
실습

1. 그리기 셀로 테스트 사진을 오른쪽으로 0, 2, 4칸 옮겨 보기
2. evaluate_shift 셀과 실습 6 셀을 실행해 exp2_cnn.pt를 불러온 CNN의 0~4칸 테스트 정확도 재기
3. MLP를 예제 5-10처럼 학습하고 저장한 best를 불러와 같은 방법으로 재고 비교하기

evaluate_shift 셀. evaluate에서 사진만 옮겨 정확도 계산

```
def evaluate_shift(model, loader, k):          # 사진을 오른쪽으로 k칸 옮겨 평가
    model.eval()
    correct = 0
    with torch.no_grad():
        for x, y in loader:
            x = torch.roll(x, shifts=k, dims=3)  # dims=3은 가로 방향
            correct += (model(x.to(device)).argmax(dim=1).cpu() == y).sum().item()
    return correct / len(loader.dataset) * 100
```

- 먼저 예측하자, 2칸 옮기면 MLP와 CNN 중 어느 쪽이 더 떨어질까?



그리기 셀의 출력, 0, 2, 4칸

- `torch.roll(x, shifts=k, dims=3)`은 사진을 오른쪽으로 k칸 민다

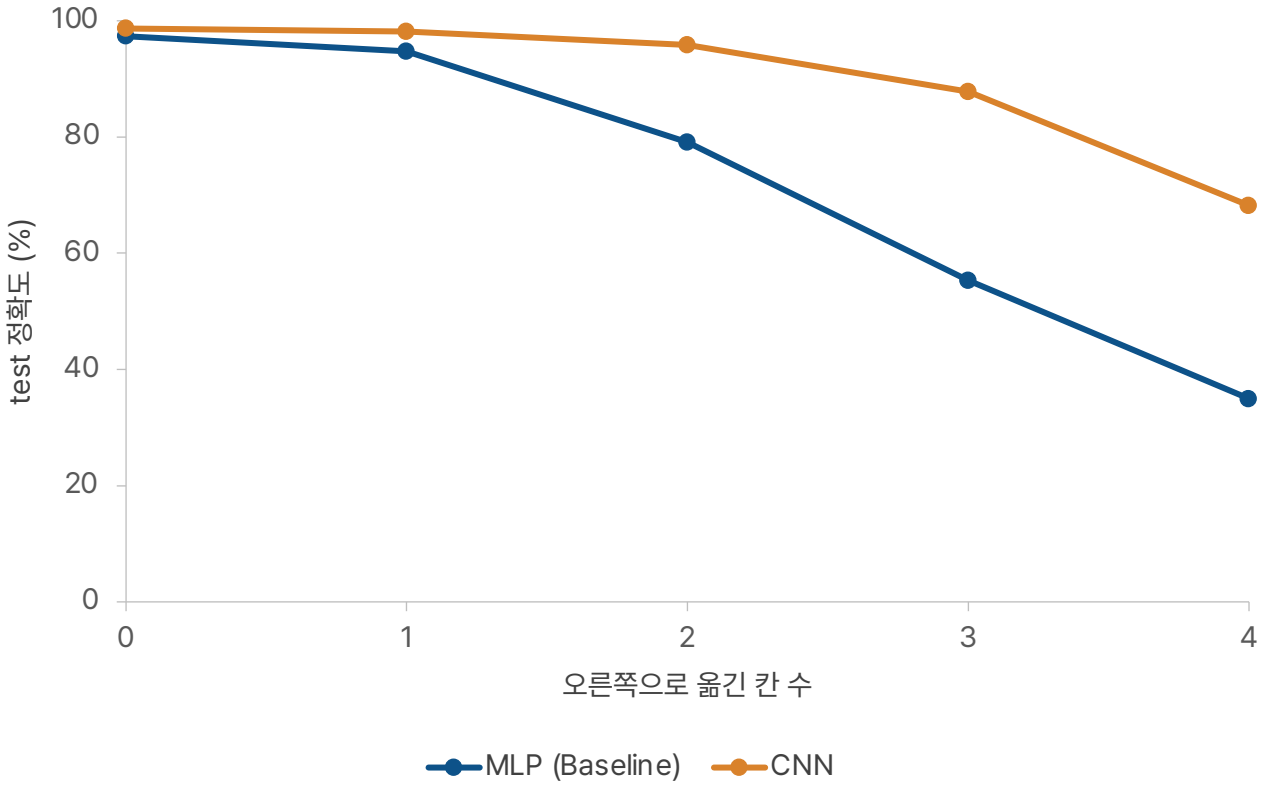
6. 실습 6: 정답 예시

실습 6. 정답 예시, 둘째의 CNN

```
model = CNN().to(device)                          # 원래 사진으로 학습한 CNN을 다시 불러온다
state = torch.load("exp2_cnn.pt", map_location=device)
model.load_state_dict(state)
for k in [0, 1, 2, 3, 4]:
    print(k, round(evaluate_shift(model, test_loader, k), 2))
# → (예) 0 98.67
# → (예) 1 98.12
# → (예) 2 95.84
# → (예) 3 87.82
# → (예) 4 68.21
```

- **실측 결과**
  - CNN은 0칸 98.67%, 2칸 **95.84%**, 4칸 68.21%다
  - 셋째의 MLP는 0칸 97.34%, 2칸 **79.13%**, 4칸 34.97%다
  - MLP는 예제 5-10에서 모델 줄과 파일 이름만 바꿔 학습하고, 저장한 best를 불러와 같은 반복문으로 잰다

6. 결과: CNN은 작은 이동에 강하다



같은 테스트 사진 10,000장을 옮겨서 잰 실측

- 2칸 옮기면 MLP는 **79.13%**로 떨어지지만 CNN은 **95.84%**를 지킨다
- 같은 kernel이 어디서나 같은 특징을 찾고(weight sharing), pooling이 작은 이동을 흡수하기 때문이다
- 4칸이면 CNN도 68.21%로 떨어진다, 다음 시간의 데이터 증강으로 보완한다

6. CNN 설계 원칙

원칙	구조에서	확인한 근거 (실측)
국소 연결	작은 kernel(3 × 3)이 이웃 픽셀만 본다	픽셀을 섞으면 CNN만 -2.96%p, MLP는 -0.06%p (실습 5)
가중치 공유	같은 kernel을 사진의 모든 위치에 쓴다	파라미터 20,490개, MLP의 약 4분의 1 (예제 5-9)
이동에 강함	가중치 공유와 pooling	2칸 옮겨도 CNN 95.84%, MLP 79.13% (실습 6)
특징의 계층	블록을 쌓을수록 receptive field가 넓어진다	3 × 3에서 10 × 10, 경계에서 획의 부분으로 (예제 5-13)

주의 CNN의 원칙은 '가까운 픽셀끼리 관련이 있다'는 사진의 성질에 맞춘 것이다, 이 가정이 깨진 데이터에서는 오히려 불리하다 (실습 5)

6. 정리, 다음 시간

오늘의 구성 요소	역할	핵심 한 줄
Conv2d (kernel)	특징 추출	kernel을 옮기며 곱하고 더해 feature map을 만든다, 입력 채널 수만큼 두껍다
padding, stride	크기 조절	출력 크기는 식 (1), 나눗셈은 버림
MaxPool2d, AvgPool2d	크기 줄이기	2×2마다 최댓값(평균) 하나, 파라미터 없음
Flatten, Linear	분류	마지막 feature map을 펴서 10개 점수
CNN	분류 모델, val 98.54%	작은 kernel로 이웃 관계를 쓰고, weight sharing과 pooling으로 작은 이동에 강하다

오늘 만든 CNN이 다음 시간부터의 새 기준이다

- **다음 시간에는 train과 val의 차이로 과적합을 진단하고 Dropout, BatchNorm, 데이터 증강으로 줄인다**